

剧本杀游戏平台 — 项目架构文档

目录

- 1. 项目概述
- 2. 技术栈
- 3. 系统架构总览
- 4. 后端模块详解
 - 4.1 实体层 (Entity)
 - 4.2 数据访问层 (Repository)
 - 4.3 业务逻辑层 (Service)
 - 4.4 控制器层 (Controller)
 - 4.5 安全模块 (Security)
 - 4.6 配置模块 (Config)
 - 4.7 AI 引擎模块 (AI)
 - 4.8 DTO 层
- 5. 前端模块详解
- 6. 数据流与交互流程
- 7. 待完善事项

1. 项目概述

本项目是一个**在线多人剧本杀游戏平台**，支持玩家创建房间、选择剧本和角色、实时对话，并由 AI 驱动的主持人（DM）和 NPC 角色参与游戏。

核心特性：

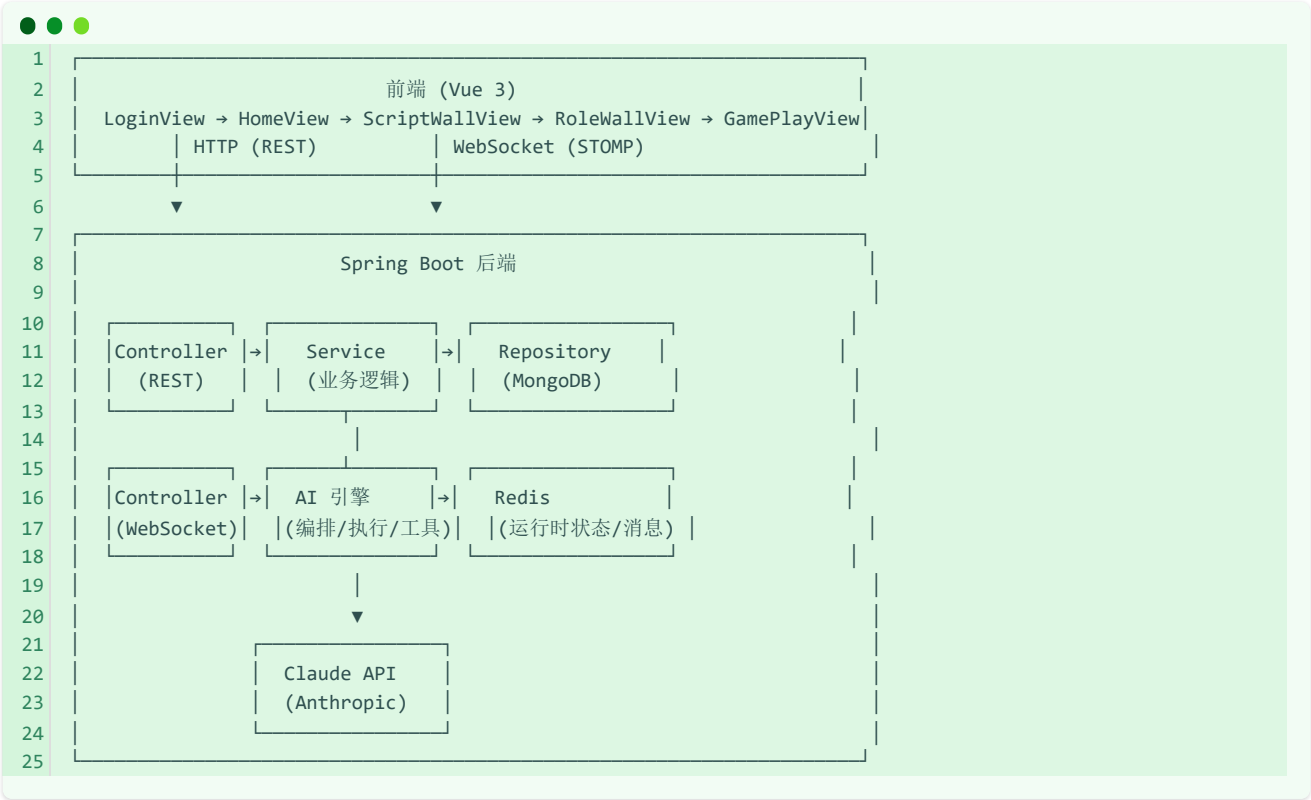
- 实时通信**：基于 WebSocket STOMP 协议的聊天和信号广播
- AI 驱动**：Spring AI + Claude 驱动的 DM（主持人）和角色代理
- 沙盒隔离**：每个 AI 角色只能看到属于自己的线索和消息，DM 拥有全局视野
- 事件驱动架构**：玩家消息触发 Spring 事件 → 编排器路由 → AI 执行器异步处理
- 双层存储**：MongoDB 持久化核心数据，Redis 管理游戏运行时状态

2. 技术栈

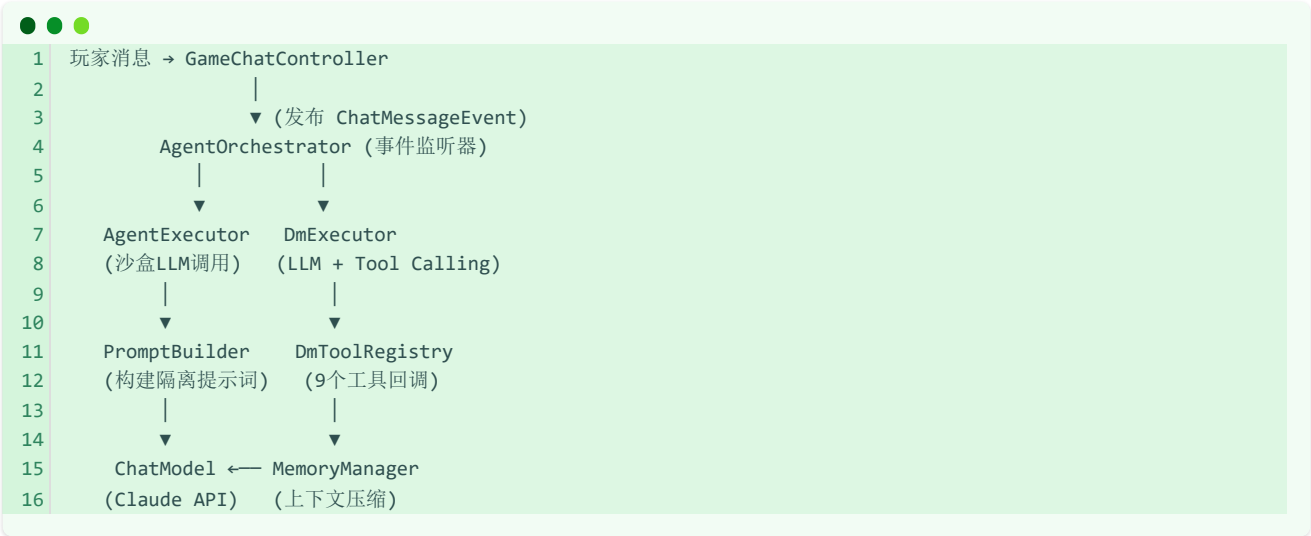
层级	技术
后端框架	Spring Boot 3.5.12, Java 17
AI 集成	Spring AI 1.1.3 (Anthropic Claude)
持久化	MongoDB (文档存储)
缓存/运行时	Redis (游戏状态、消息、JWT黑名单)
实时通信	WebSocket STOMP (SockJS 降级)
安全	Spring Security + JWT (Access + Refresh Token)

层级	技术
前端	Vue 3 + TypeScript + Element Plus + Pinia
构建	Maven (后端), Vite (前端)

3. 系统架构总览



AI 引擎内部架构



4. 后端模块详解

4.1 实体层 (Entity)

4.1.1 枚举 (entity/enums/)

枚举	值	说明
ClueType	TEXT, IMAGE, VIDEO	线索类型：文本、图片、视频
GameRoomStatus	WAITING, PLAYING, CURRENT_STAGE, FINISHED	房间生命周期状态
PhaseType	TURN_BASED, FREE_CHAT, VOTE	游戏阶段类型：轮流发言、自由讨论、投票
ScriptDifficulty	EASY(1), NORMAL(2), HARD(3), EXPERT(4)	剧本难度等级

4.1.2 MongoDB 文档实体 (entity/mongo/)

User — 用户账号（集合： `users` ）

- 字段： `id` , `username` (唯一索引), `password` (BCrypt加密), `nickname` , `avatarUrl` , `createdAt`
- 用途：用户注册和登录认证

Script — 剧本模板（集合： `scripts` ）

- 字段： `id` , `title` , `description` , `difficulty` , `playerCount` , `coverImage` , `version`
- 嵌套文档： `roles` (List<Role>), `clues` (List<Clue>), `stages` (List<ScriptStage>)
- 特殊字段： `dmConfig` — DM 系统提示词的自定义覆盖（可选，为空则使用 classpath 默认模板）
- 用途：存储剧本的所有静态数据，包括角色定义、线索、幕次内容

Role — 角色定义（嵌入 Script 中）

- 字段： `id` , `name` , `avatar` , `isNpc` (是否NPC), `prompt` (AI 角色提示词), `secret` (角色秘密)
- 搜证相关： `selfClueIds` (初始线索), `locationTag` (可访问地点), `searchPower` (搜证次数)
- 用途：定义每个角色的属性、秘密和 AI 行为指令

Clue — 线索模板（嵌入 Script 中）

- 字段： `id` , `title` , `type` (ClueType), `content` , `imageUrl` , `isInitialHidden`
- 搜证规则： `searchableRoleIds` (可搜到此线索的角色列表), `locationTag` (线索所在地点列表)
- 用途：定义游戏中的线索及其发现规则

ScriptStage — 幕次定义（嵌入 Script 中）

- 字段： `stageNumber` , `stageTitle` , `contentMap` (Map<角色ID, 剧情内容>), `audioUrl` , `unlockClueIds`
- 阶段管理： `phases` (List<StagePhase>) — 每幕包含的阶段序列
- 用途：定义每一幕的剧情文本和阶段流程

StagePhase — 阶段定义（嵌入 ScriptStage 中）

- 字段： `phaseId` , `type` (PhaseType), `speakOrder` (发言顺序列表), `timeLimitSeconds` , `dmInstruction`
- 用途：定义每个阶段的类型（轮流发言/自由讨论/投票）和规则

GameRoom — 游戏房间持久化记录（集合： `game_rooms` ）

- 字段： `roomId` , `scriptId` , `status` , `currentStage` , `members` (List<Member>), `startTime` , `endTime`
- 用途：MongoDB 中的房间持久化副本，仅在里程碑事件（创建/开始/结束）时更新

Member — 房间成员（嵌入 GameRoom/LiveGameRoom）

- 字段: `userId` (玩家为ObjectId, NPC为null), `roleId`, `isAi`, `isDm`, `isOnline`, `description`
- 用途: 表示房间中的每个参与者 (真人玩家或 AI 角色)

GameRecord — 游戏记录 (集合: `game_records`)

- 字段: `recordId`, `roomId` (索引), `scriptTitle`, `fullChatLog` (聊天记录), `startTime`, `endTime`
- 可扩展: `winnerUserIds`, `winnerRoleIds`, `aiSummary`
- 用途: 游戏结束后持久化的完整游戏记录

4.1.3 Redis 运行时实体 (`entity/redis/`)

LiveGameRoom — 游戏运行时状态

- 键格式: `game:room:{roomId}`, TTL 12小时 (活跃), 2小时 (空闲)
- 核心字段: `roomId`, `scriptId`, `status`, `currentStage`, `members`, `clueInstances`
- AI 引擎字段: `currentPhaseIndex` (当前阶段索引), `currentSpeakerRoleId` (当前发言者), `activeVote` (进行中的投票)
- 用途: 游戏进行中的唯一真实来源 (Source of Truth)

GameMessage — 聊天消息

- 键格式: Redis List `game:messages:{roomId}`
- 字段: `messageId`, `senderRoleId`, `isAi`, `senderRoleName`, `content`, `receiverRoleIds`, `timestamp`
- 用途: 存储游戏中的所有聊天消息 (包括 AI 消息)

VoteSession — 投票会话 (嵌入 LiveGameRoom)

- 字段: `voteId`, `title`, `options`, `results` (Map<角色ID, 选项>), `deadline`
- 用途: 管理进行中的投票状态

GameClueInstance — 线索运行时状态 (嵌入 LiveGameRoom)

- 字段: `clueId`, `ownerRoleIds` (持有者列表), `isPublic`, `isFound`, `discoveredAt`
- 用途: 追踪游戏中线索的发现和分发状态

VoteRecord — 投票记录

- 键格式: Redis List `game:{roomId}:votes`
- 字段: `voterUserId`, `stageNumber`, `votedRoleId`, `voteCategory`, `voteReason`, `voteWeight`
- 用途: 持久化每次投票的详细记录, 供 AI 复盘使用

4.2 数据访问层 (Repository)

接口	继承	关键方法	说明
<code>UserRepository</code>	<code>MongoRepository<User, ObjectId></code>	<code>findByUsername()</code> , <code>existsByUsername()</code>	用户查询
<code>ScriptRepository</code>	<code>MongoRepository<Script, ObjectId></code>	<code>findRandomScripts(count)</code> (聚合管道), <code>findScriptById()</code>	剧本查询, 支持随机抽取
<code>GameRecordRepository</code>	<code>MongoRepository<GameRecord, ObjectId></code>	<code>findByRoomId()</code>	游戏记录查询

注意: `GameRoomRepository` 已被删除。运行时房间状态完全由 `LiveGameRoomService` 通过 Redis 管理, MongoDB 中的 `GameRoom` 仅作归档。

4.3 业务逻辑层 (Service)

认证与用户

UserService — 用户管理（实现 Spring Security 的 **UserDetailsService**）

- **loadUserByUsername()** — Spring Security 认证入口
- **findByUsername()**, **findById()**, **existsByUsername()**, **save()** — CRUD 操作

AuthService — 认证业务逻辑

- **login()** — 验证凭据，生成 Access + Refresh Token 对
- **register()** — BCrypt 加密密码，创建用户
- **logout()** — Access Token 加入黑名单，删除 Refresh Token
- **refresh()** — 验证 Refresh Token，生成新 Token 对
- **changePassword()** — 验证旧密码，更新并使所有 Token 失效
- **getUserInfo()** — 返回当前用户信息

游戏核心流程

GameRoomService — 房间生命周期管理

- **createRoom()** — 创建房间，清理用户之前的活跃房间
- **selectScript()** — 选择剧本，缓存到 Redis
- **selectRole()** — 选择角色，**自动为剩余空位填充 AI 成员**
- **leaveRoom()** — 离开房间，标记离线，最后一个人离开时设置空闲 TTL
- **getRoles()** — 获取可选角色列表（过滤已被选择的）
- **toDetailDTO()** — 将 LiveGameRoom 转换为前端展示 DTO

GameFlowService — 游戏流程控制

- **startGame()** — 校验状态，初始化阶段追踪（currentPhaseIndex, currentSpeakerRoleId），设置 PLAYING
- **getStageContent()** — 返回当前幕次中**当前用户角色对应的**剧情内容
- **advanceStage()** — **压缩当前幕消息**（通过 MemoryManager），推进到下一幕
- **endGame()** — 持久化游戏记录，**异步触发 AI 复盘**（FinalReviewService），清理 Redis

LiveGameRoomService — Redis 游戏状态 CRUD

- **init()**, **get()**, **save()**, **evict()** — 基本 CRUD
- **bindUserRoom()** / **unbindUserRoom()** — 用户-房间绑定（防止重复创建房间）
- **setMemberOnline()** — 更新在线状态
- **advanceStage()** — 更新幕次编号
- **getMessages()** — 获取房间所有消息

GameChatService — 消息收发

- **sendMessage()** — 存储消息到 Redis List，通过 WebSocket 广播
- **sendAiMessage()** — 与 **sendMessage** 相同逻辑，但 **isAi=true**

ScriptCacheService — 剧本 Redis 缓存

- **getScript()** — Redis 缓存优先，未命中则从 MongoDB 加载并缓存（TTL 24h）
- **cacheScript()**, **evictScript()** — 缓存管理

游戏扩展服务

PhaseTimerService — 阶段计时器

- **startTurnTimer()** — 发言超时后通知 DM 推进到下一位
- **startFreeChatTimer()** — 自由讨论超时后通知 DM 结束讨论

- `startVoteTimer()` — 投票超时后广播关闭信号
- `cancelTimer()` — 取消当前计时器
- 内部使用 `ScheduledExecutorService` (2线程) 和 `ConcurrentHashMap` 管理

`VoteService` — 投票管理

- `castVote()` — 记录投票, 广播进度, 返回是否全员已投票
- `closeVote()` — 关闭投票, 广播结果, 清除 `LiveGameRoom` 中的 `activeVote`

`GameSummaryService` (接口) / `DefaultGameSummaryService` ** (默认实现)

- `summarize()` — 当前为桩实现, 直接返回原始消息列表
- 用途: 游戏结束时生成聊天日志摘要

4.4 控制器层 (Controller)

REST 控制器

`AuthController` (`/api/auth`)

端点	方法	认证	说明
<code>/login</code>	POST	否	登录, 返回 Token 对
<code>/register</code>	POST	否	注册
<code>/logout</code>	POST	是	登出, 使 Token 失效
<code>/password</code>	PUT	是	修改密码
<code>/refresh</code>	POST	否	刷新 Token
<code>/info</code>	GET	是	获取用户信息

`ScriptController` (`/api/scripts`)

端点	方法	说明
<code>/random</code>	GET	随机获取剧本列表 (默认8个)

`GameRoomController` (`/api/rooms`)

端点	方法	说明
<code>/</code>	POST	创建房间
<code>/ {roomId}</code>	GET	获取房间详情
<code>/ {roomId} /script</code>	PUT	选择剧本
<code>/ {roomId} /roles</code>	GET	获取可选角色
<code>/ {roomId} /role</code>	PUT	选择角色
<code>/ {roomId} /start</code>	POST	开始游戏
<code>/ {roomId} /stage</code>	GET	获取当前幕内容
<code>/ {roomId} /stage /advance</code>	POST	推进下一幕
<code>/ {roomId} /leave</code>	POST	离开房间

WebSocket 控制器

GameChatController — STOMP 消息处理

- **@MessageMapping("/chat.{roomId}")** — 接收聊天消息
- 流程：认证校验 → 成员校验 → 游戏状态校验 → 发送消息 → **发布 ChatMessageEvent** (触发 AI 引擎)

4.5 安全模块 (Security)

JwtProperties — JWT 配置属性

- **secret** — 签名密钥 (Base64 编码)
- **accessTokenExpiration** — Access Token 有效期 (默认 7200 秒 = 2 小时)
- **refreshTokenExpiration** — Refresh Token 有效期 (默认 2592000 秒 = 30 天)

JwtService — JWT 核心服务

- Token 生成：Access Token (含 userId + username)、Refresh Token (含 userId)
- Token 验证：签名校验 + 过期检查 + Redis 黑名单检查
- 黑名单机制：登出时将 Access Token 的 JTI 存入 Redis (剩余有效期为 TTL)

JwtAuthenticationFilter — HTTP 请求过滤器

- 拦截所有 HTTP 请求 (公开路径除外：**/api/auth/login**, **/api/auth/register**, **/api/auth/refresh**)
- 从 **Authorization: Bearer <token>** 提取并验证 Token
- 验证通过后将 Claims 设置到 SecurityContext

WebSocketAuthInterceptor — WebSocket 认证拦截器

- 拦截 STOMP CONNECT 帧
- 从 STOMP 头部的 **Authorization** 字段验证 JWT
- 认证失败抛出 **MessageDeliveryException**

4.6 配置模块 (Config)

SecurityConfig — Spring Security 配置

- 无状态会话 (STATELESS), 禁用 CSRF
- 公开路径: **/api/auth/****, **/ws/****, **/ws-sockjs/****
- CORS: 允许 **localhost:***, 支持 GET/POST/PUT/DELETE/OPTIONS
- JWT 过滤器注册在 **UsernamePasswordAuthenticationFilter** 之前

WebSocketConfig — WebSocket STOMP 配置

- 消息代理: **/topic** (广播), **/queue** (点对点)
- 应用前缀: **/app**
- 端点: **/ws** (原生 WebSocket), **/ws-sockjs** (SockJS 降级)
- 入站通道拦截器: **WebSocketAuthInterceptor**

JacksonConfig — JSON 序列化配置

- **ObjectId** → 十六进制字符串
- **LocalDateTime** → ISO-8601 格式

AsyncConfig — 异步执行器

- Bean 名称: **aiExecutor**
- 核心线程: 4, 最大线程: 16, 队列容量: 100
- 线程名前缀: **ai-engine-**

AiEngineProperties — AI 引擎配置

- **charsPerToken** (3.5) — Token 估算比例 (中文约 3.5 字符/Token)
- **tokenThresholdRatio** (0.7) — 上下文窗口 70% 时触发压缩
- **maxContextTokens** (100000) — 最大上下文 Token 数
- **slidingWindowSize** (20) — 最近消息滑动窗口大小
- **turnTimeoutSeconds** (60) — 发言超时
- **voteTimeoutSeconds** (120) — 投票超时
- **freeChatTimeoutSeconds** (300) — 自由讨论超时

DataInitializer — 开发环境数据初始化

- **@Profile("dev")** — 仅在开发环境生效
- 启动时若 scripts 集合为空, 插入 10 个测试剧本

4.7 AI 引擎模块 (AI)

4.7.1 事件 (ai/event/)

事件类	触发场景	字段
ChatMessageEvent	玩家发送消息	roomId, senderRoleId, content, fromAi
AgentReplyEvent	需要 AI 角色回复	roomId, roleId
DmRequestEvent	需要 DM 介入	roomId, triggerRoleId, reason

4.7.2 编排器 (ai/orchestrator/)

AgentOrchestrator — 消息路由核心

- **@EventListener onChatMessage(ChatMessageEvent)** — 监听所有玩家消息
- 路由规则:
 1. **AI 消息**: 直接忽略 (防止循环)
 2. **DM 请求检测**: 包含 **@DM** / **@主持人**、搜证/投票关键词 → 转发给 DmExecutor
 3. **轮流发言阶段**: 推进到下一个发言者, 如果是 AI 则触发 AgentExecutor; 全部发言完毕则请 DM 决定下一步
 4. **自由讨论阶段**: 检查 @提及 → 触发对应 AI 角色; 无提及 → 请 DM 选择回复者
 5. **投票阶段**: 不触发 AI 回复

4.7.3 执行器 (ai/executor/)

AgentExecutor — AI 角色执行器

- **@Async("aiExecutor") executeAgentReply(roomId, roleId)**
- 流程:
 1. 广播 TYPING 信号
 2. 通过 PromptBuilder 构建 **沙盒提示词** (仅可见线索和消息)
 3. 调用 ChatModel (Claude API)
 4. 通过 GameChatService 发送 AI 消息
 5. 广播 TYPING_END 信号

DmExecutor — DM 执行器

- **@Async("aiExecutor") executeDmAction(roomId, triggerRoleId, reason)**
- 流程:
 1. 广播 TYPING 信号

- 2. 构建工具上下文 (DmToolContext)
- 3. 通过 DmToolRegistry 生成 ToolCallback 列表
- 4. 通过 PromptBuilder 构建 **全权限 DM 提示词**
- 5. 使用 **ToolCallingChatOptions** 调用 ChatModel (LLM 可自主调用工具)
- 6. 发送 DM 叙述性回复
- 7. 广播 TYPING_END 信号

4.7.4 提示词构建 (ai/prompt/)

PromptBuilder — 提示词模板引擎

- **buildDmPrompt()** — DM 提示词 (全局视野: 所有秘密、线索、搜证能力表)
- **buildAgentPrompt()** — 角色提示词 (沙盒隔离: 仅可见线索、仅可见消息)
- **buildCompressionPrompt()** — 压缩提示词 (将一幕的消息总结为结构化 JSON)
- **buildReviewPrompt()** — 复盘提示词 (全局数据 → 评分和叙事)
- **estimateTokens()** — Token 数估算 (**text.length() / charsPerToken**)

提示词模板文件位于 **src/main/resources/prompts/** :

- **dm-system.md** — DM 系统提示词模板
- **agent-system.md** — AI 角色系统提示词模板
- **compression.md** — 上下文压缩提示词模板
- **review-system.md** — 游戏复盘提示词模板

沙盒隔离机制 :

- DM 可见: 所有角色秘密、所有线索、完整消息流
- Agent 仅可见: 自己的秘密、 **ownerRoleIds** 包含自己的线索、 **receiverRoleIds** 包含自己或为公开的消息

4.7.5 DM 工具系统 (ai/tool/)

DmTool (接口) — 工具定义协议

- **name()** — 工具名称 (LLM 通过此名称调用)
- **description()** — 工具描述 (告诉 LLM 何时使用)
- **inputType()** — 输入参数类型 (Java Class)
- **execute(input, ctx)** — 执行逻辑

DmToolContext — 工具执行上下文

- 字段: **room** (LiveGameRoom), **script** (Script), **currentPhaseId** , **triggerRoleId**

DmToolRegistry — 工具注册中心

- 通过 Spring DI 自动发现所有 **DmTool** 实现
- **buildCallbacks(ctx)** — 将所有工具封装为 Spring AI **FunctionToolCallback**
- **可扩展**: 新增工具只需实现 **DmTool** 接口并添加 **@Component** 注解

9 个内置工具实现 (ai/tool/impl/):

工具	名称	功能	副作用
AuthorizeSearchTool	authorizeSearch	授权角色在指定地点搜证	扣除搜证次数、创建线索实例、广播 CLUE_FOUND
InitiateVoteTool	initiateVote	发起投票	创建 VoteSession、广播 VOTE_OPEN
ReadFullScriptTool	readFullScript	DM 查阅剧本真相	无 (只读)
AssignTurnTool	assignTurn	指定下一个发言者	更新 currentSpeakerRoleId、广播 TURN_CHANGE

工具	名称	功能	副作用
<code>AdvancePhaseTool</code>	<code>advancePhase</code>	推进到下一阶段	更新 <code>currentPhaseIndex</code> 、广播 <code>PHASE_CHANGE</code> 或 <code>STAGE_COMPLETE</code>
<code>SelectRespondentsTool</code>	<code>selectRespondents</code>	选择回复者	无 (返回建议)
<code>DecidePhaseTransitionTool</code>	<code>decidePhaseTransition</code>	决定阶段转换	无 (返回决策)
<code>EndFreeChatTool</code>	<code>endFreeChat</code>	结束自由讨论	广播 <code>PHASE_CHANGE(VOTE_CHECK)</code>
<code>SkipVoteTool</code>	<code>skipVote</code>	跳过投票	无 (返回 <code>ADVANCE_STAGE</code>)

4.7.6 记忆管理 (`ai/memory/`)

`MemoryManager` — 上下文压缩与检索

- `compressStage()` — 异步调用 LLM 将一幕的消息压缩为 `StageSummary`，存入 Redis
- `shouldCompress()` — 判断当前提示词是否超过 Token 阈值 (70%)
- `getMemoryFragments()` — 检索已压缩的历史幕次摘要
- `getRecentMessages()` — 返回最近 N 条消息 (滑动窗口)
- `extractJson()` — 从 LLM 回复中提取 JSON (处理 markdown 代码块)

`StageSummary` — 压缩输出结构

- `keyEvents` — 关键事件列表 (类型、发起者、目标、摘要)
- `relationshipChanges` — 角色关系变化
- `unresolved` — 未解之谜列表

4.7.7 游戏复盘 (`ai/review/`)

`FinalReviewService` — AI 复盘生成

- `@Async("aiExecutor") generateReview()` — 在游戏结束时异步调用
- 流程: 收集所有记忆片段 + 线索池摘要 + 投票记录 → 调用 LLM → 解析为 `GameReviewResult` → 广播到房间

`GameReviewResult` — 复盘结果结构

- `narrative` — 叙事总结
- `truthReveal` — 真相揭示
- `roleScores` — 每个角色的评分、亮点和错过的线索
- `unresolvedMysteries` — 未解之谜
- `mvp` — 最佳表现者

4.8 DTO 层

DTO	用途
<code>LoginRequest</code>	登录请求 (username, password)
<code>RegisterRequest</code>	注册请求 (username, password, nickname)
<code>TokenResponse</code>	Token 响应 (accessToken, refreshToken, expiresIn)
<code>ChangePasswordRequest</code>	改密请求 (oldPassword, newPassword)
<code>UserInfoResponse</code>	用户信息响应
<code>RefreshTokenRequest</code>	刷新 Token 请求
<code>ChatMessageRequest</code>	WebSocket 聊天消息请求
<code>ChatMessageDTO</code>	聊天消息广播 DTO

DTO	用途
RoleDTO	角色列表展示 (含 isAvailable)
ScriptSummaryDTO	剧本摘要展示
RoomDetailDTO	房间详情展示 (含 MemberDTO 列表)
StageContentDTO	幕次内容展示

5. 前端模块详解

5.1 技术架构

```
1 Vue 3 + TypeScript + Vite
2 |— Element Plus (UI 组件库)
3 |— Pinia (状态管理)
4 |— Vue Router (路由)
5 |— Axios (HTTP 客户端)
6 |— STOMP.js (WebSocket 客户端)
```

5.2 路由 (router/index.ts)

路径	视图	认证	说明
/login	LoginView	否	登录/注册页
/home	HomeView	是	首页, 创建房间
/profile	ProfileView	是	个人资料
/game/:roomId/scripts	ScriptWallView	是	剧本选择墙
/game/:roomId/roles	RoleWallView	是	角色选择墙
/game/:roomId/play	GamePlayView	是	游戏主界面

路由守卫: 检查 useAuthStore().isLoggedIn, 未登录跳转 /login

5.3 状态管理 (stores/)

useAuthStore — 认证状态

- 状态: accessToken, refreshToken, userInfo
- 持久化到 localStorage
- 方法: setTokens(), setUserInfo(), clearAuth(), isLoggedIn (getter)

useGameStore — 游戏状态

- 状态: roomId, scriptId, myRoleId, messages (ChatMessage[]), gameStatus
- 方法: setRoom(), setScript(), setMyRole(), addMessage(), setGameStatus(), clearGame()

5.4 API 层 (api/)

axios.ts — Axios 实例配置

- 基础 URL: /api
- 请求拦截器: 自动附加 Authorization: Bearer <token> 头
- 响应拦截器: 401 时自动刷新 Token, 排队等待刷新完成后重试

API 模块：

- `auth.ts` — login, register, logout, refreshToken, getUserInfo, changePassword
- `room.ts` — createRoom, getRoom, selectScript, getRoles, selectRole, startGame, leaveRoom
- `script.ts` — getRandomScripts

5.5 WebSocket (`composables/useWebSocket.ts`)

- `connect(roomId, onMessage)` — 建立 STOMP 连接, 订阅 `/topic/room.{roomId}`
- `send(roomId, content)` — 发送消息到 `/app/chat.{roomId}`
- `disconnect()` — 断开连接
- `connected` — 连接状态响应式引用

5.6 视图 (`views/`)

视图	功能
LoginView	登录/注册表单, 双标签页切换
HomeView	侧边栏菜单, 创建房间按钮
ScriptWallView	4x2 网格展示剧本, 点击选择
RoleWallView	自适应网格展示角色, 点击选择
GamePlayView	游戏主界面: 聊天室、消息气泡、退出按钮
ProfileView	个人资料展示、密码修改

6. 数据流与交互流程

6.1 完整游戏生命周期

1

1. 用户登录 → AuthService 生成 JWT

2

2. 创建房间 → GameRoomService.createRoom() → Redis LiveGameRoom

3

3. 选择剧本 → ScriptCacheService 缓存到 Redis

4

4. 选择角色 → 自动为空位填充 AI + DM 成员

5

5. 开始游戏 → status=PLAYING, 初始化阶段追踪

6

6. 游戏循环:

7

a. 玩家发消息 → WebSocket → GameChatController → 发布 ChatMessageEvent

8

b. AgentOrchestrator 监听事件:

9

- DM 请求 → DmExecutor (带工具调用)

10

- 轮流发言 → 推进发言者, AI 角色触发 AgentExecutor

11

- 自由讨论 → @提及触发对应 AI, 否则 DM 选择回复者

12

c. AI 回复 → GameChatService.sendAiMessage → WebSocket 广播

13

d. DM 可调用工具: 搜证授权、投票发起、阶段推进等

14

7. 幕次推进 → MemoryManager 压缩当前幕消息 → 推进到下一幕

15

8. 游戏结束 → 持久化记录 → FinalReviewService AI 复盘 → 清理 Redis

6.2 WebSocket 信号类型

信号	频道	说明
TYPING / TYPING_END	.signal	AI 正在输入/输入结束
TURN_CHANGE	.signal	发言权转移
PHASE_CHANGE	.signal	阶段切换
STAGE_COMPLETE	.signal	当前幕完成

信号	频道	说明
VOTE_OPEN	.signal	投票开始
VOTE_UPDATE	.signal	投票进度更新
VOTE_CLOSED	.signal	投票结束
CLUE_FOUND	主频道	线索被发现
GAME_END	主频道	游戏结束 + 复盘数据
SYSTEM	主频道	系统消息

7. 待完善事项

7.1 高优先级

1. 前端游戏信号处理

- 当前 `GamePlayView` 仅处理聊天消息
- 缺少对 `.signal` 频道的订阅和 UI 响应 (TYPING 指示器、TURN_CHANGE 高亮、VOTE 弹窗、PHASE_CHANGE 过渡动画)
- 建议: 在 `useWebSocket` 中增加信号频道订阅, 在 `GamePlayView` 中添加对应 UI 组件

2. 前端阶段内容展示

- `GamePlayView` 缺少幕间剧情阅读界面
- 当游戏开始或推进幕次时, 应调用 `GET /api/rooms/{roomId}/stage` 并展示 `StageContentDTO`
- 建议: 增加剧情阅读模态框/全屏页面

3. 投票 UI

- 前端没有投票界面组件
- 需要: 投票弹窗 (选项列表、投票按钮)、投票进度展示、投票结果展示
- 后端需补充: 投票 REST 端点 (`POST /api/rooms/{roomId}/vote`)

4. 搜证 UI

- 前端没有搜证交互界面
- 需要: 地点选择界面、搜证结果展示、已发现线索列表
- 当前搜证完全由 AI DM 通过工具执行, 但玩家需要 UI 来查看自己的线索

5. `DefaultGameSummaryService` 仍为桩实现

- 当前直接返回原始消息列表
- 建议: 集成 AI 摘要或直接移除该接口, 复盘功能已由 `FinalReviewService` 覆盖

7.2 中优先级

6. `AgentOrchestrator` 路由增强

- 当前自由讨论阶段每条消息都触发 DM 选择回复者 (LLM 调用开销大)
- 建议: 增加节流/去抖逻辑, 例如 5 秒内多条消息合并处理
- 可增加规则引擎: 仅对疑问句或特定关键词触发 AI 回复

7. `PhaseTimerService` 与 `AgentOrchestrator` 集成

- 计时器已实现但尚未在任何地方启动
- 需要在 `AgentOrchestrator` 或 `GameFlowService` 中, 当进入新阶段时启动对应计时器
- TURN_BASED → `startTurnTimer()`; FREE_CHAT → `startFreeChatTimer()`; VOTE → `startVoteTimer()`

8. VoteService 与前端集成

- `VoteService` 已实现但无 REST 端点
- 需要在 `GameRoomController` 中增加 `POST /rooms/{roomId}/vote` 端点
- 需要处理投票结束后的流程（DM 决定是否推进）

9. 错误处理与重试

- `AgentExecutor` 和 `DmExecutor` 捕获异常后仅记录日志
- 建议：增加重试机制（限制次数），向玩家广播友好的错误消息
- AI API 限流处理：增加指数退避重试

10. MemoryManager 压缩触发完善

- `shouldCompress()` 方法已实现但 `advanceStage` 中直接调用 `compressStage`，未使用阈值判断
- 建议：在 `AgentExecutor` / `DmExecutor` 构建提示词前检查 `shouldCompress()`，超阈值时主动触发压缩

7.3 低优先级

11. 游戏房间邀请/加入

- 当前没有多人加入同一房间的机制
- 需要：房间码/邀请链接、加入房间 API、成员列表实时同步

12. 断线重连

- WebSocket 断线后无自动重连和消息补偿
- 建议：前端增加重连逻辑，后端增加历史消息拉取 API（`GET /api/rooms/{roomId}/messages`）

13. 游戏记录回放

- `GameRecord` 已持久化但没有查看入口
- 需要：历史游戏列表页、游戏记录详情页、复盘结果展示

14. 集成测试

- 缺少 Spring Boot 集成测试（`@SpringBootTest` + 嵌入式 Redis/MongoDB + Mock ChatModel）
- `StriptKillGameDemo2ApplicationTests.contextLoads` 因缺少测试配置而失败
- 建议：创建 `application-test.properties` 和 `TestAiConfig`（Mock ChatModel）

15. 前端线索详情组件

- 玩家无法查看已获得的线索详情
- 需要：线索列表面板（侧边栏或弹窗），展示线索标题、内容、发现时间

16. AI 模型配置界面

- `application.properties` 中的 AI 配置（模型、温度等）目前只能手动修改
- 设计文档中提到需支持 API 修改配置
- 建议：增加管理员 API 端点动态调整 AI 参数

17. 国际化

- 所有提示词模板和错误消息使用硬编码中文
- 如需支持多语言，需要引入消息国际化（`MessageSource`）

18. 监控与可观测性

- 缺少 AI 调用的指标监控（调用次数、延迟、Token 消耗、错误率）
- 建议：集成 Spring Boot Actuator + Micrometer，记录 AI 引擎核心指标